Multi-Agent Research Pipeline Documentation

1. Introduction

This document provides complete technical documentation for building a Multi-Agent Research Pipeline using LangGraph, LangChain, OpenAI LLMs, and the Tavily Search API. The system is designed to automate research tasks including planning, searching, and summarizing information into a cohesive final output.

2. System Overview

The pipeline follows a modular, agent-based architecture similar to industry-grade research automation tools. The system takes a research topic as input, breaks it into sub-questions, performs web research, and produces a final structured summary. Execution Flow: User Input → Planner Agent → Searcher Agent → Writer Agent → Final Output

3. Agent Descriptions

Planner Agent: - Breaks the input topic into three detailed, actionable sub-questions. - Defines structured tasks for subsequent agents. - Ensures clarity and completeness before passing data forward. Searcher Agent: - Uses Tavily Search API to retrieve reliable information. - Summarizes retrieved data using OpenAI LLM. - Produces short factual responses for each task. Writer Agent: - Synthesizes results from the searcher. - Produces a cohesive research summary with introduction, insights, and conclusion.

4. Architecture Using LangGraph

LangGraph is used to create a state-based, multi-node execution pipeline. Graph Structure: - Node 1: PLANNER - Node 2: SEARCHER - Node 3: WRITER - END: Final node returning the output State Model: - topic: str - tasks: List[str] - search_results: Dict[str, str] - final_summary: str

5. Execution Pipeline

Step-by-step: 1. User provides a research topic. 2. Planner Agent generates three sub-questions. 3. Searcher Agent fetches and summarizes information using Tavily + LLM. 4. Writer Agent synthesizes the content into the final research summary. 5. Pipeline returns the final results.

6. Code Structure Overview

The system includes: - Robust OpenAI API wrapper with retries - Tavily search integration - LangGraph workflow with nodes for each agent - Error handling for missing keys, bad input, or search issues Main sections of code: - State model definition - Planner, Searcher, Writer agent implementations - LangGraph workflow compilation - Runner with user input

7. Installation

Install required packages: pip install openai tavily-python langgraph pydantic tenacity Set environment variables: export OPENAI_API_KEY="your-key" export TAVILY_API_KEY="your-key"

8. Example Usage

Input: "Impact of Artificial Intelligence in Healthcare" Planner Output: 1. Current applications of AI in healthcare 2. Benefits and risks of AI-based healthcare systems 3. Future trends and advancements Searcher Output: - Summaries retrieved via Tavily + LLM Writer Output: - A cohesive 4–6 paragraph report summarizing all findings

9. Advantages of This System

- Fully modular agent design - Easy to extend with new agents - Real-time data from Tavily Search - Reliable summarization via OpenAI LLMs - Professional research workflow similar to enterprise tools

10. Troubleshooting

Common Issues: - Missing API keys → Set OPENAI_API_KEY and TAVILY_API_KEY - Planner returns incorrect format → Fallback parsing handles this - Tavily returns empty → The system marks missing data gracefully - OpenAI rate limits → Retries handled via Tenacity

11. Conclusion

This system is a complete and scalable architecture for automated research generation. It demonstrates modular multi-agent design, real-time information retrieval, and structured writing using LLMs. The pipeline can be expanded for academic research, document generation, data extraction, and enterprise AI systems.